Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №4

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для прогнозирования временного ряда: влияние размера окна и числа скрытых нейронов»

Выполнил:

Студент 4 курса

Группы ИИ-26

Петручик Д. А.

Проверил:

Никитин С. К.

Брест 2026

Цель работы: Научиться формировать обучающую выборку методом скользящего окна, нормализовать данные, оценивать качество одношагового и многошагового прогноза и исследовать влияние размера окна p и числа нейронов скрытого слоя N на точность прогноза и скорость обучения.

Ход работы:

1. Сгенерировать ряд по формуле варианта и построить его график с отметкой границы между обучающим и тестовым участками.
2. Нормализовать ряд по минимуму и максимуму обучающего участка: x_norm = (x - x_min) / (x_max - x_min). Записать формулу обратного пересчёта и использовать её при вычислении всех метрик в исходной шкале. Объяснить, почему min/max нельзя брать по всему ряду.
3. Сформировать обучающую и тестовую выборки методом скользящего окна размера p (вход – p значений, цель – следующее значение). Указать количество получившихся примеров в каждой выборке.
4. Реализовать многослойный персептрон p–N–1:
    - скрытый слой – сигмоида;
    - выходной слой – линейный (задача регрессии), f(s) = s;
    - функция потерь – MSE;
    - обучение – онлайн-режим, перемешивание порядка примеров в каждой эпохе;
    - вывести формулы градиентов для линейного выходного слоя и сигмоидного скрытого слоя и привести их в отчёте.
5. Критерий остановки: средняя по обучающей выборке ошибка $E_s \le E_e$, где $E_e = 10^{-4}$ (нормализованная шкала), либо достижение 20 000 эпох. Скорость обучения подобрать самостоятельно из диапазона 0.01–0.3 и обосновать выбор.
6. **Базовый эксперимент.** Обучить сеть с параметрами p и N из своего варианта. Зафиксировать число эпох, итоговую Es, а также метрики на обучающем и тестовом участках **в исходной шкале**: MAE, RMSE и MAPE (если ряд проходит через ноль – вместо MAPE использовать sMAPE, и пояснить почему).
7. **Исследование окна.** При фиксированном N из варианта обучить сеть для $p \in \{2, 4, p_{вар}, 2 \cdot p_{вар}\}$. Для каждого p выполнить по 5 запусков с разными seed и свести в таблицу: среднее и разброс числа эпох, среднее и разброс тестового MAE, число запусков, не достигших критерия остановки.
8. **Исследование скрытого слоя.** При фиксированном p из варианта обучить сеть для $N \in \{1, 2, N_{вар}, 2 \cdot N_{вар}\}$. Выполнить ту же серию из 5 запусков и заполнить аналогичную таблицу.
9. **Многошаговый прогноз.** Для лучшей по тестовому MAE конфигурации выполнить рекурсивный прогноз на 30 шагов вперёд, начиная с k = 140: каждое предсказание подставляется в окно как входное значение для следующего шага. Построить график «эталон – рекурсивный прогноз» и график абсолютной ошибки в зависимости от номера шага. Определить, на каком шаге ошибка превышает 5 % от размаха ряда.
10. Построить графики:
    - ряд с границей train/test и одношаговым прогнозом сети на обоих участках на одних осях;
    - сходимость (Es от эпохи) для базовой конфигурации и для двух конфигураций из исследования окна на одних осях (логарифмическая шкала по Y);
    - диаграмма рассеяния «эталон – прогноз» на тестовом участке с диагональю y = x;
    - рекурсивный прогноз и рост ошибки по шагам (п. 9).

11. Реализовать режим функционирования: пользователь вводит p последовательных значений ряда в исходной шкале (либо номер k, с которого брать окно из ряда). Программа выводит:

- нормализованные входы;
- прогноз в нормализованной шкале (4 знака после запятой);
- прогноз в исходной шкале;
- истинное значение x(k) и абсолютную ошибку (если окно взято из ряда).

Продемонстрировать работу не менее чем на 3 окнах из обучающего участка, 3 окнах из тестового участка и 1–2 пользовательских окнах (в том числе с «выбросом»).

12. Вставить в отчет фото/скан рукописных расчётов. Вверху листа от руки должны быть написаны Ваша Фамилия, Инициалы, номер группы, вариант и дата выполнения:

- Графическая схема пайплайна (Блок-схема). Составьте корректную цепочку обработки данных, используя предложенный набор блоков (приложение 1). Объясните свой выбор.
- Ручной расчёт 1 шага обучения. На отдельном листе бумаги от руки распишите один полный шаг (Forward + Backward pass) для сети 3–2–1 со следующими параметрами, привязанными к вашему варианту V:
  - Входы: X = [0.2, 0.5, 0.8], Цель: d = 0.6
  - Веса скрытого слоя: W11 = 0.05*V, W12 = -0.2, W21 = 0.3, W22 = 0.05*V, W31 = -0.1, W32 = 0.4 (b1=0.1, b2=-0.1)
  - Веса выходного слоя: W_out1 = 0.5, W_out2 = -0.2*V (b_out = 0.2)

Вариант	a	b	c	d	p	N
10	2.2	0.27	1.1	0.62	8	5

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

A, B, C, D = 2.2, 0.27, 1.1, 0.62
P, N = 8, 5
K_TOTAL = 200
K_TRAIN = 140
Lr = 0.1
Ee = 1e-4
MAX_EPOCHS = 20000
SEED = 42


def generate_series(a, b, c, d, n = K_TOTAL):
    k = np.arange(n)
    return a * np.cos(b * k) + c * np.sin(d * k)

print("Сеть 8–5–1, 132 обучающих и 60 тестовых примеров.")
print("Ряд пересекает ноль: используем sMAPE вместо MAPE.")
series = generate_series(A, B, C, D)

plt.figure(figsize=(11, 4))
plt.plot(series, label="x(k)")
plt.axvline(K_TRAIN - 0.5, color="r", linestyle="--", label="граница train/test")
plt.xlabel("k"); plt.ylabel("x(k)"); plt.title("Ряд x(k)")
plt.legend(); plt.grid(alpha=0.3)
plt.show()


def fit_minmax(x_train):
    return x_train.min(), x_train.max()

def normalize(x, x_min, x_max):
    return (x - x_min) / (x_max - x_min)
```

```
def denormalize(xn, x_min, x_max):
    return xn * (x_max - x_min) + x_min

x_min, x_max = fit_minmax(series[:K_TRAIN])
series_n = normalize(series, x_min, x_max)
print(f"x_min = {x_min:.4f}, x_max = {x_max:.4f}")
print(f"train в [{series_n[:K_TRAIN].min():.3f}; {series_n[:K_TRAIN].max():.3f}]")
print(f"test в [{series_n[K_TRAIN:].min():.3f}; {series_n[K_TRAIN:].max():.3f}]")


def make_windows(x, p, t_start, t_end):
    X = np.array([x[t - p:t] for t in range(t_start, t_end)])
    y = np.array([x[t] for t in range(t_start, t_end)])
    return X, y

Xtr, ytr = make_windows(series_n, P, P, K_TRAIN)
Xte, yte = make_windows(series_n, P, K_TRAIN, K_TOTAL)
print("train:", Xtr.shape, ytr.shape, "-> примеров:", len(Xtr))
print("test:", Xte.shape, yte.shape, "-> примеров:", len(Xte))


def sigmoid(s):
    return 1.0 / (1.0 + np.exp(-np.clip(s, -700, 700)))

def init_weights(p, n, rng, scale=0.3):
    return {"W1": rng.uniform(-scale, scale, (n, p)),
            "b1": rng.uniform(-scale, scale, n),
            "W2": rng.uniform(-scale, scale, n),
            "b2": float(rng.uniform(-scale, scale))}

def forward(w, x):
    h = sigmoid(x @ w["W1"].T + w["b1"])
    y = h @ w["W2"] + w["b2"]
    return h, y

def backward(w, x, h, y, d):
    delta_out = 2.0 * (y - d)
    dW2 = delta_out * h
    db2 = delta_out
    delta_h = delta_out * w["W2"] * h * (1.0 - h)
    dW1 = np.outer(delta_h, x)
    db1 = delta_h
    return dW1, db1, dW2, db2

def mse(w, X, d):
    _, y = forward(w, X)
    return float(np.mean((y - d) ** 2))

def train(Xtr, ytr, p, n, seed, lr = Lr, ee = Ee, max_epochs = MAX_EPOCHS):
    rng = np.random.default_rng(seed)
    w = init_weights(p, n, rng)
    history = []
    for epoch in range(1, max_epochs + 1):
        for i in rng.permutation(len(Xtr)):
            x, d = Xtr[i], ytr[i]
            h, y = forward(w, x)
            dW1, db1, dW2, db2 = backward(w, x, h, y, d)
            w["W1"] -= lr * dW1
            w["b1"] -= lr * db1
            w["W2"] -= lr * dW2
            w["b2"] -= lr * db2
        Es = mse(w, Xtr, ytr)
        history.append(Es)
        if Es <= ee:
            return w, epoch, history, True
    return w, max_epochs, history, False


def mae(y, yh):   return float(np.mean(np.abs(y - yh)))
def rmse(y, yh):  return float(np.sqrt(np.mean((y - yh) ** 2)))
def smape(y, yh):
    denominator = np.abs(y) + np.abs(yh)
```

```
    terms = np.divide(2 * np.abs(y - yh), denominator,
                      out=np.zeros_like(denominator), where=denominator > 0)
    return float(100 * np.mean(terms))

def predict_orig(w, Xn, x_min, x_max):
    _, yn = forward(w, Xn)
    return denormalize(yn, x_min, x_max)

def evaluate(w, Xn, y_true, x_min, x_max):
    yh = predict_orig(w, Xn, x_min, x_max)
    return {"MAE": mae(y_true, yh), "RMSE": rmse(y_true, yh),
            "sMAPE": smape(y_true, yh)}

RUN_LR_STUDY = False
if RUN_LR_STUDY:
    print(" lr   | эпохи (ср ± σ) | test MAE | не сошлись")
    for lr in [0.01, 0.03, 0.05, 0.1, 0.2, 0.3]:
        eps, maes, fails = [], [], 0
        for s in [1, 2, 3, 4, 5]:
            w, ep, hist, ok = train(Xtr, ytr, P, N, s, lr = lr)
            eps.append(ep)
            maes.append(evaluate(w, Xte, series[K_TRAIN:], x_min, x_max)["MAE"])
            fails += (not ok)
        print(f"{lr:5.2f} | {np.mean(eps):6.1f} ± {np.std(eps):5.1f} | {np.mean(maes):.4f}   |
{fails}")



w_base, ep_base, hist_base, ok_base = train(Xtr, ytr, P, N, SEED)
print(f"Эпох: {ep_base}, Es = {hist_base[-1]:.3e}, критерий достигнут: {ok_base}")
for name, X, y_true in [("train", Xtr, series[P:K_TRAIN]),
                        ("test ", Xte, series[K_TRAIN:])]:
    m = evaluate(w_base, X, y_true, x_min, x_max)
    print(f"{name}: MAE={m['MAE']:.4f}  RMSE={m['RMSE']:.4f}  "
          f"sMAPE={m['sMAPE']:.2f}%")


SEEDS = [1, 2, 3, 4, 5]

def run_config(p, n, seeds=SEEDS):
    Xtr_, ytr_ = make_windows(series_n, p, p, K_TRAIN)
    Xte_, _ = make_windows(series_n, p, K_TRAIN, K_TOTAL)
    runs = []
    for s in seeds:
        print(f"Обучение: p={p}, N={n}, seed={s}", flush=True)
        w, ep, hist, ok = train(Xtr_, ytr_, p, n, s)
        m = evaluate(w, Xte_, series[K_TRAIN:], x_min, x_max)
        runs.append({"p": p, "n": n, "seed": s, "w": w, "epochs": ep,
                     "hist": hist, "ok": ok, "test_mae": m["MAE"]})
    return runs

def print_table(title, name, table):
    print(title)
    print(f"{name:>4} | эпохи: ср. ± σ  | test MAE: ср. ± σ | не достигли Es≤Ee (из 5)")
    for key, runs in table.items():
        ep = np.array([r["epochs"] for r in runs])
        ma = np.array([r["test_mae"] for r in runs])
        fails = sum(not r["ok"] for r in runs)
        print(f"{key:>4} | {ep.mean():8.1f} ± {ep.std():6.1f} | {ma.mean():.4f} ± {ma.std():.4f}  |
{fails}")
    print()

window_runs = {p: run_config(p, N) for p in [2, 4, P, 2 * P]}
neuron_runs = {n: (window_runs[P] if n == N else run_config(P, n))
               for n in [1, 2, N, 2 * N]}

print_table(f"Исследование окна (N = {N})", "p", window_runs)
print_table(f"Исследование скрытого слоя (p = {P})", "N", neuron_runs)

configs = {}
for p_, runs in window_runs.items():
    configs[(p_, N)] = runs
for n_, runs in neuron_runs.items():
```

```
    configs[(P, n_)] = runs
best_key = min(configs, key=lambda k: np.mean([r["test_mae"] for r in configs[k]]))
best_run = min(configs[best_key], key=lambda r: r["test_mae"])
print(f"Лучшая конфигурация по среднему test MAE: p = {best_key[0]}, N = {best_key[1]}")
print(f"Берём её лучший запуск: seed = {best_run['seed']}, test MAE = {best_run['test_mae']:.4f}")


def recursive_forecast(w, series_n, p, start, steps):
    window = list(series_n[start - p:start])
    out = []
    for _ in range(steps):
        _, y = forward(w, np.array(window))
        out.append(float(y))
        window = window[1:] + [float(y)]
    return np.array(out)

STEPS = 30
p_best = best_run["p"]
rec_pred = denormalize(recursive_forecast(best_run["w"], series_n, p_best, K_TRAIN, STEPS), x_min,
x_max)
rec_true = series[K_TRAIN:K_TRAIN + STEPS]
rec_err = np.abs(rec_true - rec_pred)
threshold = 0.05 * (series.max() - series.min())
bad = np.where(rec_err > threshold)[0]
print(f"Порог 5% размаха: {threshold:.4f}")
if len(bad):
    print(f"Ошибка впервые превышает порог на шаге {bad[0] + 1} (k = {K_TRAIN + bad[0]})")
else:
    print("За 30 шагов ошибка порог не превысила")


Xtr, ytr = make_windows(series_n, P, P, K_TRAIN)
Xte, yte = make_windows(series_n, P, K_TRAIN, K_TOTAL)
pred_tr = predict_orig(w_base, Xtr, x_min, x_max)
pred_te = predict_orig(w_base, Xte, x_min, x_max)

plt.figure(figsize=(12, 4))
plt.plot(series, "k", label="эталон")
plt.plot(range(P, K_TRAIN), pred_tr, "b--", label="прогноз (train)")
plt.plot(range(K_TRAIN, K_TOTAL), pred_te, "r--", label="прогноз (test)")
plt.axvline(K_TRAIN - 0.5, color="gray", linestyle=":")
plt.xlabel("k"); plt.ylabel("x(k)"); plt.title("Одношаговый прогноз, базовая сеть")
plt.legend(); plt.grid(alpha=0.3); plt.show()

plt.figure(figsize=(8, 5))
plt.semilogy(hist_base, label=f"базовая: p={P}, N={N}")
for p_ in (2, 2 * P):
    plt.semilogy(window_runs[p_][0]["hist"], label=f"p={p_}, N={N} (seed={SEEDS[0]})")
plt.axhline(Ee, color="r", linestyle=":", label="Ee = 1e-4")
plt.xlabel("эпоха"); plt.ylabel("Es (лог. шкала)"); plt.title("Сходимость")
plt.legend(); plt.grid(alpha=0.3, which="both"); plt.show()

plt.figure(figsize=(5.5, 5.5))
plt.scatter(series[K_TRAIN:], pred_te, s=18)
lims = [series.min() - 0.2, series.max() + 0.2]
plt.plot(lims, lims, "r--", label="y = x")
plt.xlabel("эталон"); plt.ylabel("прогноз"); plt.title("Тест: эталон vs прогноз")
plt.legend(); plt.grid(alpha=0.3); plt.show()

fig, ax = plt.subplots(1, 2, figsize=(13, 4))
steps_axis = np.arange(1, STEPS + 1)
ax[0].plot(steps_axis, rec_true, "k", label="эталон")
ax[0].plot(steps_axis, rec_pred, "r--", label="рекурсивный прогноз")
ax[0].set_xlabel("шаг"); ax[0].set_ylabel("x"); ax[0].legend(); ax[0].grid(alpha=0.3)
ax[1].plot(steps_axis, rec_err, "o-")
ax[1].axhline(threshold, color="r", linestyle="--", label="5% размаха")
ax[1].set_xlabel("шаг"); ax[1].set_ylabel("|ошибка|"); ax[1].legend(); ax[1].grid(alpha=0.3)
plt.show()


def run_mode(w, p, series, x_min, x_max, k=None, window=None, title=""):
    if window is None:
        window = series[k - p:k]
```

```
    window = np.asarray(window, dtype=float)
    if window.shape != (p,) or not np.all(np.isfinite(window)):
        raise ValueError(f"Нужно ровно {p} конечных чисел.")
    xn = normalize(window, x_min, x_max)
    _, yn = forward(w, xn)
    y = denormalize(yn, x_min, x_max)
    print(f"--- {title}")
    print("окно (исходная шкала):", np.round(window, 4))
    print("нормализованные входы:", np.round(xn, 4))
    print(f"прогноз (норм.):   {float(yn):.4f}")
    print(f"прогноз (исходн.): {float(y):.4f}")
    if k is not None:
        print(f"истинное x({k}) = {series[k]:.4f}, абс. ошибка = {abs(series[k] - y):.4f}")

for k in [20, 70, 120]:
    run_mode(w_base, P, series, x_min, x_max, k=k, title=f"train, k={k}")
for k in [145, 170, 195]:
    run_mode(w_base, P, series, x_min, x_max, k=k, title=f"test, k={k}")

outlier = series[100 - P:100].copy()
outlier[3] = 8.0
run_mode(w_base, P, series, x_min, x_max, k=100, window=outlier, title="окно k=100 с выбросом 8.0")
run_mode(w_base, P, series, x_min, x_max, window=[1.0, 1.5, 2.0, 1.2, 0.3, -0.5, -1.0, -1.5],
title="пользовательское окно")

def interactive_mode(w, p, series, x_min, x_max):
    print("\nРежим функционирования. Команды:")
    print(f"k 150 — прогноз x(150) по предыдущим {p} значениям")
    print(f"w <{p} чисел через пробел> — собственное окно")
    print("q — выход. Десятичный разделитель — точка.")
    while True:
        try:
            line = input("> ").strip()
        except (EOFError, KeyboardInterrupt):
            break
        if line.lower() in ("q", "exit", "выход"):
            break
        try:
            parts = line.split()
            if not parts:
                continue
            if parts[0].lower() == "k" and len(parts) == 2:
                k = int(parts[1])
                if not p <= k < len(series):
                    raise ValueError(f"Допустимые k: {p}…{len(series)-1}")
                run_mode(w, p, series, x_min, x_max, k=k, title=f"k={k}")
            elif parts[0].lower() == "w":
                window = [float(v) for v in parts[1:]]
                run_mode(w, p, series, x_min, x_max, window=window,
                         title="Пользовательское окно")
            else:
                print("Введите k <индекс>, w <числа> или q.")
        except ValueError as exc:
            print("Ошибка ввода:", exc)


if __name__ == "__main__":
    interactive_mode(w_base, P, series, x_min, x_max)
```

Результат тестирования:

```
Сеть 8-5-1, 132 обучающих и 60 тестовых примеров.
Ряд пересекает ноль: используем sMAPE вместо MAPE.
x_min = -3.2819, x_max = 3.2858
train в [0.000; 1.000]
test в [0.025; 0.986]
train: (132, 8) (132,) -> примеров: 132
test: (60, 8) (60,) -> примеров: 60
```

Эпох: 74, Es = 9.697e-05, критерий достигнут: True
train: MAE=0.0548 RMSE=0.0647 sMAPE=8.98%
test : MAE=0.0585 RMSE=0.0678 sMAPE=16.99%
Обучение: p=2, N=5, seed=1
Обучение: p=2, N=5, seed=2
Обучение: p=2, N=5, seed=3
Обучение: p=2, N=5, seed=4
Обучение: p=2, N=5, seed=5
Обучение: p=4, N=5, seed=1
Обучение: p=4, N=5, seed=2
Обучение: p=4, N=5, seed=3
Обучение: p=4, N=5, seed=4
Обучение: p=4, N=5, seed=5
Обучение: p=8, N=5, seed=1
Обучение: p=8, N=5, seed=2
Обучение: p=8, N=5, seed=3
Обучение: p=8, N=5, seed=4
Обучение: p=8, N=5, seed=5
Обучение: p=16, N=5, seed=1
Обучение: p=16, N=5, seed=2
Обучение: p=16, N=5, seed=3
Обучение: p=16, N=5, seed=4
Обучение: p=16, N=5, seed=5
Обучение: p=8, N=1, seed=1
Обучение: p=8, N=1, seed=2
Обучение: p=8, N=1, seed=3
Обучение: p=8, N=1, seed=4
Обучение: p=8, N=1, seed=5
Обучение: p=8, N=2, seed=1
Обучение: p=8, N=2, seed=2
Обучение: p=8, N=2, seed=3
Обучение: p=8, N=2, seed=4
Обучение: p=8, N=2, seed=5
Обучение: p=8, N=10, seed=1
Обучение: p=8, N=10, seed=2
Обучение: p=8, N=10, seed=3
Обучение: p=8, N=10, seed=4
Обучение: p=8, N=10, seed=5
Исследование окна (N = 5)

p	эпохи: ср. ± σ	test MAE: ср. ± σ	не достигли Es≤Ee (из 5)
2	20000.0 ± 0.0	0.1726 ± 0.0045	5
4	3740.4 ± 174.3	0.0558 ± 0.0004	0
8	66.6 ± 2.5	0.0564 ± 0.0024	0
16	30.8 ± 8.9	0.0488 ± 0.0022	0

Исследование скрытого слоя (p = 8)

N	эпохи: ср. ± σ	test MAE: ср. ± σ	не достигли Es≤Ee (из 5)
1	78.6 ± 3.0	0.0571 ± 0.0033	0

```
   2 |      71.8 ±    3.2 | 0.0563 ± 0.0028  | 0
   5 |      66.6 ±    2.5 | 0.0564 ± 0.0024  | 0
  10 |      69.2 ±    4.8 | 0.0537 ± 0.0035  | 0

Лучшая конфигурация по среднему test MAE: p = 16, N = 5
Берём её лучший запуск: seed = 5, test MAE = 0.0453
Порог 5% размаха: 0.3284
За 30 шагов ошибка порог не превысила
--- train, k=20
окно (исходная шкала): [-1.1823 -0.9756 -1.0211 -1.2161 -1.364  -1.2564 -0.7616
0.1137]
нормализованные входы: [0.3197 0.3512 0.3442 0.3145 0.292  0.3084 0.3837 0.517 ]
прогноз (норм.):   0.6728
прогноз (исходн.): 1.1366
истинное x(20) = 1.2142, абс. ошибка = 0.0775
--- train, k=70
окно (исходная шкала): [-0.3863  0.4917  1.0115  1.1557  1.0463  0.8873  0.8762
1.121 ]
нормализованные входы: [0.4409 0.5746 0.6537 0.6757 0.659  0.6348 0.6331 0.6704]
прогноз (норм.):   0.7299
прогноз (исходн.): 1.5118
истинное x(70) = 1.5923, абс. ошибка = 0.0805
--- train, k=120
окно (исходная шкала): [1.1975 2.2488 2.8699 2.9533 2.5532 1.8592 1.1227 0.5638]
нормализованные входы: [0.682  0.8421 0.9367 0.9494 0.8885 0.7828 0.6706 0.5855]
прогноз (норм.):   0.5344
прогноз (исходн.): 0.2282
истинное x(120) = 0.2934, абс. ошибка = 0.0653
--- test, k=145
окно (исходная шкала): [1.5413 1.252  1.0937 1.1783 1.4805 1.8463 2.0535 1.9005]
нормализованные входы: [0.7344 0.6903 0.6662 0.6791 0.7251 0.7808 0.8124 0.7891]
прогноз (норм.):   0.7114
прогноз (исходн.): 1.3901
истинное x(145) = 1.2909, абс. ошибка = 0.0993
--- test, k=170
окно (исходная шкала): [ 2.0358  2.7543  3.1068  2.9334  2.224   1.1282 -0.093  -
1.153 ]
нормализованные входы: [0.8097 0.9191 0.9727 0.9463 0.8383 0.6715 0.4855 0.3241]
прогноз (норм.):   0.2260
прогноз (исходн.): -1.7977
истинное x(170) = -1.8345, абс. ошибка = 0.0368
--- test, k=195
окно (исходная шкала): [ 2.4687  1.5893  0.6988  0.0241 -0.3225 -0.3756 -0.2913 -
0.2768]
нормализованные входы: [0.8756 0.7417 0.6061 0.5034 0.4506 0.4425 0.4554 0.4576]
прогноз (норм.):   0.4374
прогноз (исходн.): -0.4095
истинное x(195) = -0.5005, абс. ошибка = 0.0911
--- окно k=100 с выбросом 8.0
```

```
окно (исходная шкала): [ 2.6254  3.1854  3.2189  8.      1.7391  0.6026 -0.4343 -
1.1504]
нормализованные входы: [0.8994 0.9847 0.9898 1.7178 0.7645 0.5915 0.4336 0.3245]
прогноз (норм.):   0.2886
прогноз (исходн.): -1.3865
истинное x(100) = -1.4558, абс. ошибка = 0.0693
--- пользовательское окно
окно (исходная шкала): [ 1.   1.5  2.   1.2  0.3 -0.5 -1.  -1.5]
нормализованные входы: [0.652  0.7281 0.8042 0.6824 0.5454 0.4236 0.3474 0.2713]
прогноз (норм.):   0.3388
прогноз (исходн.): -1.0567
```

График с отметкой границы между обучающим и тестовым участками:

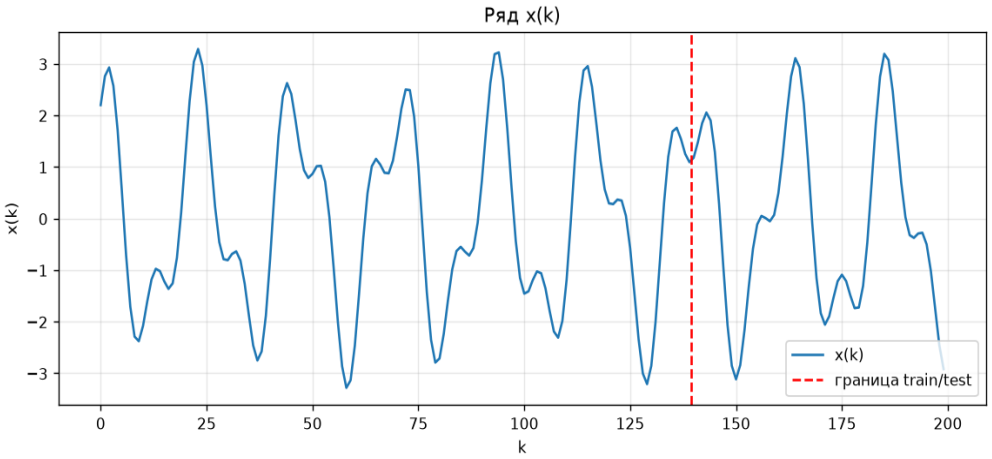
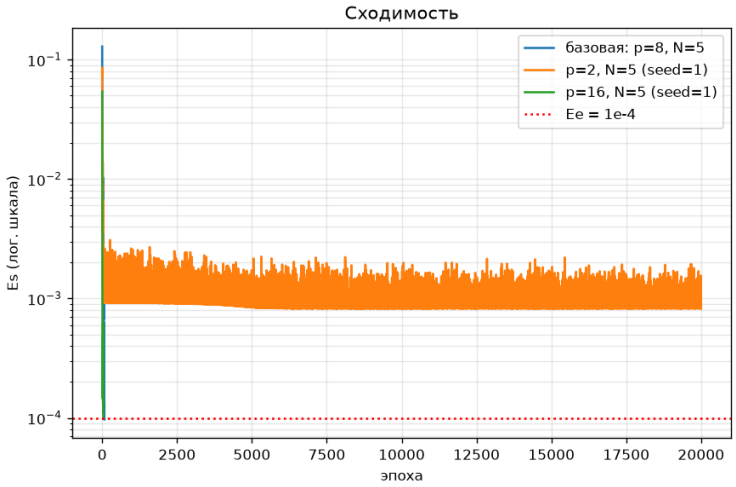


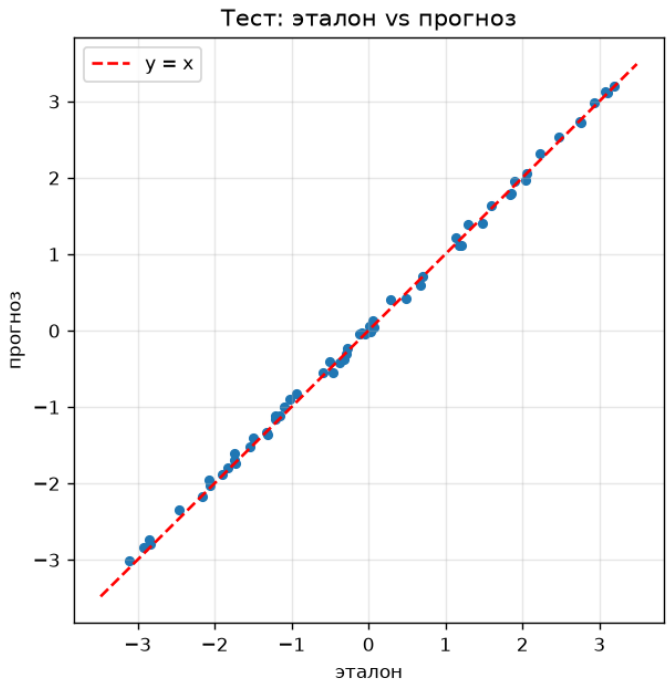
График с одношаговым прогнозом:



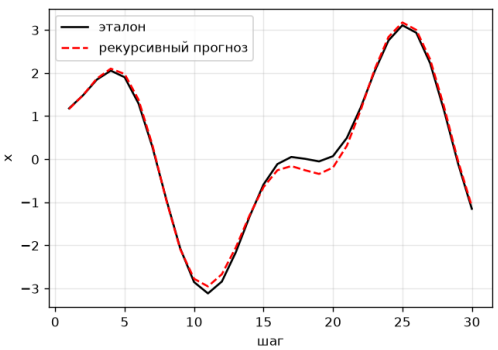
Сходимость:

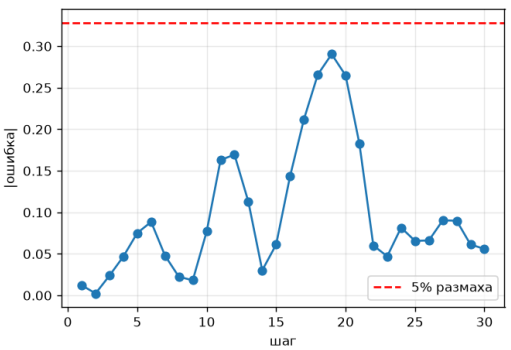


Тест «Эталон – прогноз»:



Рекурсивный прогноз и рост ошибки по шагам:





Ручной расчет одного шага обучения сети 3-2-1 при V = 10:

Дано: x = [0.2; 0.5; 0.8], d = 0.6

Скрытый слой: W11 = 0.5, W12 = -0.2, W21 = 0.3, W22 = 0.5, W31 = -0.1, W32 = 0.4, b1 = 0.1, b2 = -0.1

Выходной слой: W_out1 = 0.5, W_out2 = -2.0, b_out = 0.2

Forward

$s1 = 0.2 \cdot 0.5 + 0.5 \cdot 0.3 + 0.8 \cdot (-0.1) + 0.1 = 0.10 + 0.15 - 0.08 + 0.1 = 0.27$

$s2 = 0.2 \cdot (-0.2) + 0.5 \cdot 0.5 + 0.8 \cdot 0.4 + (-0.1) = -0.04 + 0.25 + 0.32 - 0.1 = 0.43$

$h1 = 1 / (1 + e^{(-0.27)}) = 1 / 1.7634 \approx 0.5671$

$h2 = 1 / (1 + e^{(-0.43)}) = 1 / 1.6505 \approx 0.6059$

$y = 0.5 \cdot 0.5671 + (-2.0) \cdot 0.6059 + 0.2 \approx -0.7282$ (линейный выход)

$E = (0.6 - (-0.7282))^2 = 1.3282^2 \approx 1.7641$

Backward

$\delta_{out} = 2(y - d) = 2(-0.7282 - 0.6) = -2.6564$

Выходной слой:

$dW_{out1} = \delta_{out} \cdot h1 = -2.6564 \cdot 0.5671 \approx -1.5064$

$dW_{out2} = \delta_{out} \cdot h2 = -2.6564 \cdot 0.6059 \approx -1.6094$

$db_{out} = \delta_{out} = -2.6564$

Скрытый слой (берём старые W_out):

$\delta1 = \delta_{out} \cdot W_{out1} \cdot h1(1 - h1)$

$= -2.6564 \cdot 0.5 \cdot 0.5671 \cdot 0.4329 \approx -0.3261$

$\delta2 = \delta_{out} \cdot W_{out2} \cdot h2(1 - h2)$

$= -2.6564 \cdot (-2.0) \cdot 0.6059 \cdot 0.3941 \approx 1.2686$

$dW11 = 0.2 \cdot \delta1 \approx -0.0652$

$dW12 = 0.2 \cdot \delta2 \approx 0.2537$

$dW21 = 0.5 \cdot \delta1 \approx -0.1630$

$dW22 = 0.5 \cdot \delta2 \approx 0.6343$

$dW31 = 0.8 \cdot \delta1 \approx -0.2609$

$dW32 = 0.8 \cdot \delta 2 \approx 1.0149$

$db1 = \delta 1 \approx -0.3261$

$db2 = \delta 2 \approx 1.2686$

Обновление

$w_new = w - \eta \cdot dw,\ \eta = 0.1$

$W_out1 = 0.5 + 0.15064 \approx 0.6506$

$W_out2 = -2.0 + 0.16094 \approx -1.8391$

$b_out = 0.2 + 0.26564 \approx 0.4656$

$W11 = 0.5 + 0.00652 \approx 0.5065$

$W21 = 0.3 + 0.01630 \approx 0.3163$

$W31 = -0.1 + 0.02609 \approx -0.0739$

$W12 = -0.2 - 0.02537 \approx -0.2254$

$W22 = 0.5 - 0.06343 \approx 0.4366$

$W32 = 0.4 - 0.10149 \approx 0.2985$

$b1 = 0.1 + 0.03261 \approx 0.1326$

$b2 = -0.1 - 0.12686 \approx -0.2269$

Проверка: прогон с новыми весами даёт $y \approx -0.1598$, $E \approx 0.5773 < 1.7641$.

То есть ошибка уменьшилась.

Вывод: Проделав данную лабораторную работу я научился формировать обучающую выборку методом скользящего окна, нормализовать данные, оценил качество одношагового и многошагового прогноза и исследовал влияние размера окна p и числа нейронов скрытого слоя N на точность прогноза и скорость обучения.